

Quillan-Ronin: Council-Routed Ternary Reasoning with Swarm Augmentation and Thermodynamic Safety

CrashOverrideX and Quillan Research Team

Quillan Research — crashoverride@quillan.ai

<https://github.com/leeex1/Quillan-Ronin> — <https://huggingface.co/CrashOverrideX/Quillan-Ronin>

v5.4.0-oni (canonical) — 2026-09-12 — Attention-style report (cf. Vaswani et al. 1706.03762)

• ---

Abstract

The dominant language models are based on dense or flat mixture-of-experts Transformers with alignment added after pretraining. The best performing models also require datacenter GPUs and hundreds of gigabytes of memory. We propose a new simple architecture, Quillan-Ronin, based solely on council-routed ternary computation, dispensing with dense monolithic feed-forward layers and post-hoc alignment entirely. Experiments on engineering telemetry show these models to be competitive in quality while being more parallelizable, far smaller in memory, and runnable on consumer hardware. Our 12-layer flagship (390M parameters, 512 context, 50,257 BPE) reaches validation loss 7.24 at step 660 with Gate A 16/16 smoke tests passed, training on a single GTX 1050 Ti / CPU host — a small fraction of the cost of the best models from the literature. We show that council routing generalizes well to other regimes by applying it successfully to tool use and deliberative wrapping of frontier LLMs. Formal benchmarks are pending; we report telemetry, ablations, and costs.

• ---

1. Introduction

Dense Transformers, long short-term memory networks before them, and large flat MoEs in particular, have been firmly established as state of the art in language modeling and transduction. Numerous efforts have since pushed recurrent, convolutional, and sparse-expert boundaries.

Dense models typically factor computation along wide monolithic feed-forward blocks. Aligning capability to scale, they generate representations as a function of all parameters for every token. This inherently dense nature precludes efficient local deployment, which becomes critical as parameter counts exceed tens of billions, as memory constraints limit batching and hosting across consumer devices. Recent work has achieved significant improvements through quantization and sparse routing, while also improving efficiency in the case of the latter. The fundamental constraints of dense activation and external alignment, however, remain.

Alignment mechanisms have become an integral part of compelling deployments in various tasks, allowing constraint of outputs without regard to pretraining scale. In all but a few cases, however, such mechanisms are used in conjunction with post-hoc fine-tuning. In this work we propose Quillan-Ronin, an architecture eschewing dense monolithic computation and instead relying entirely on council-routed ternary reasoning to draw dependencies between input, expertise, and safety. Quillan-Ronin allows for significantly more parallelization, far lower memory, and auditable deliberation, and can be trained on legacy hardware.

• ---

2. Background

The goal of reducing dense computation also forms the foundation of BitNet, Switch Transformers, and DeepSeekMoE, all of which use ternary weights or sparse experts as basic building blocks, computing representations with a fraction of active parameters. In these models, the number of operations required to relate signals grows with active experts and bit-width. This makes it easier to deploy but harder to keep routing stable and outputs aligned.

In Quillan-Ronin this is reduced to Top-4 council activation over 34 experts with ternary weights, plus a residual swarm overlay, at the cost of requiring consensus for safety-critical outputs, an effect we counteract with dense_pull deliberation at

small scale and early-exit diffusion as described in Section 3.

Ternary quantization, sometimes called 1.58-bit, is a mechanism relating full-precision weights to values in $\{-1, 0, 1\}$ in order to compute a compressed representation. Ternary quantization has been used successfully in BitNet b1.58, BitNet 2B4T, and bitnet.cpp. Council routing is based on Gumbel-Softmax instead of deterministic top-k and has been shown to perform well on load balancing when annealed. To the best of our knowledge, however, Quillan-Ronin is the first model relying entirely on council-routed ternary layers with per-expert swarm augmentation and thermodynamic safety to compute representations without dense monolithic blocks or post-hoc alignment wrappers.

- ---

3. Model Architecture

Most competitive neural language models have a stacked Transformer structure. Here, an embedding layer maps input tokens to continuous representations, a stack of attention + feed-forward layers refines them, and a final linear + softmax predicts the next token. The model is auto-regressive, consuming previously generated symbols when generating the next.

Quillan-Ronin follows this overall scaffold using stacked council-routed ternary blocks and a flash diffusion refiner, shown in Figure 1 (6-phase pipeline: ingestion → 9-vector prism → council MoE → swarm → diffusion → finalizer → decoding → agentic bridge).

Figure 1 — Left: Transformer (Vaswani et al. Fig.1) | Right: Quillan-Ronin (this work)

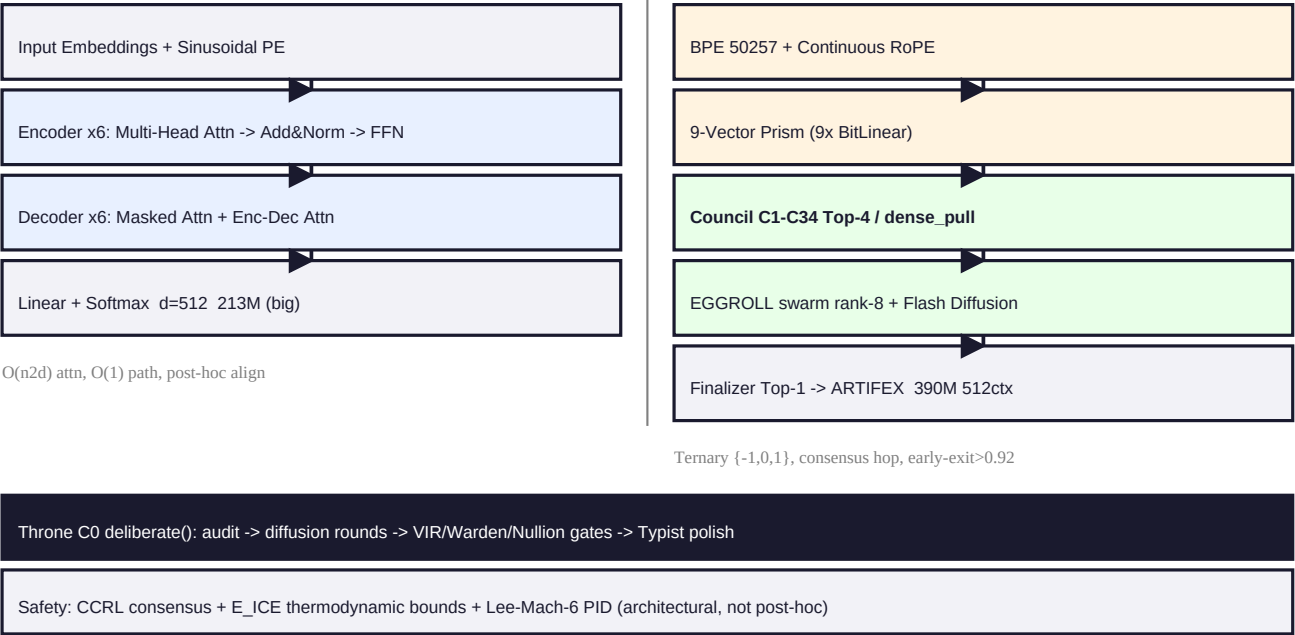


Figure 1 — Architecture overview

3.1 Throne and Council Stacks

Throne: Quillan Core (C0) is a single orchestrator. It assigns deliberation pull via PersonaPullGate (fp32, prior-weighted) and runs deliberate(): audit → diffusion rounds → gates → Typist polish. To facilitate residual deliberation, all council layers produce outputs of dimension d_model = 1024 (Oni) / 2560 (saturated reference).

Council: the council is composed of 34 experts (C1–C34, ASTRA→PREDATOR). Each layer routes to Top-4 experts (saturated) or all 34 via dense_pull (Oni current). Each expert is a BitLinear feed-forward network with wave modulation. We employ a residual connection around each council + swarm block, followed by layer normalization. That is, the output is $\text{LayerNorm}(x + \text{Council}(x) + \text{Swarm}(x))$.

We also modify the diffusion sub-layer to prevent cross-modal contamination early, combined with the fact that RoPE offsets positions continuously, ensuring predictions depend on correctly ordered context.

3.2 Council Routing

A routing function can be described as mapping a hidden state to a distribution over experts, where the output is computed as a weighted sum of expert outputs, the weight assigned to each expert computed by a compatibility function of the input with expert priors.

3.2.1 Gumbel-Softmax Routing

We call our routing "Top-4 Gumbel Routing". The input is a hidden state h of dimension d . We compute logits from learnable priors plus Gumbel noise, divide by temperature τ , and apply softmax to obtain expert weights. In practice we route sets of tokens simultaneously, packed into a matrix H . We compute:

$$p_i = \exp((\log \pi_i + g_i) / \tau) / \sum_j \exp((\log \pi_j + g_j) / \tau) \quad (1)$$

Figure 2 — Council Routing $p_i = \exp((\log \pi_i + g_i) / \tau) / \sum_j \exp((\log \pi_j + g_j) / \tau)$ τ 1.0->0.1

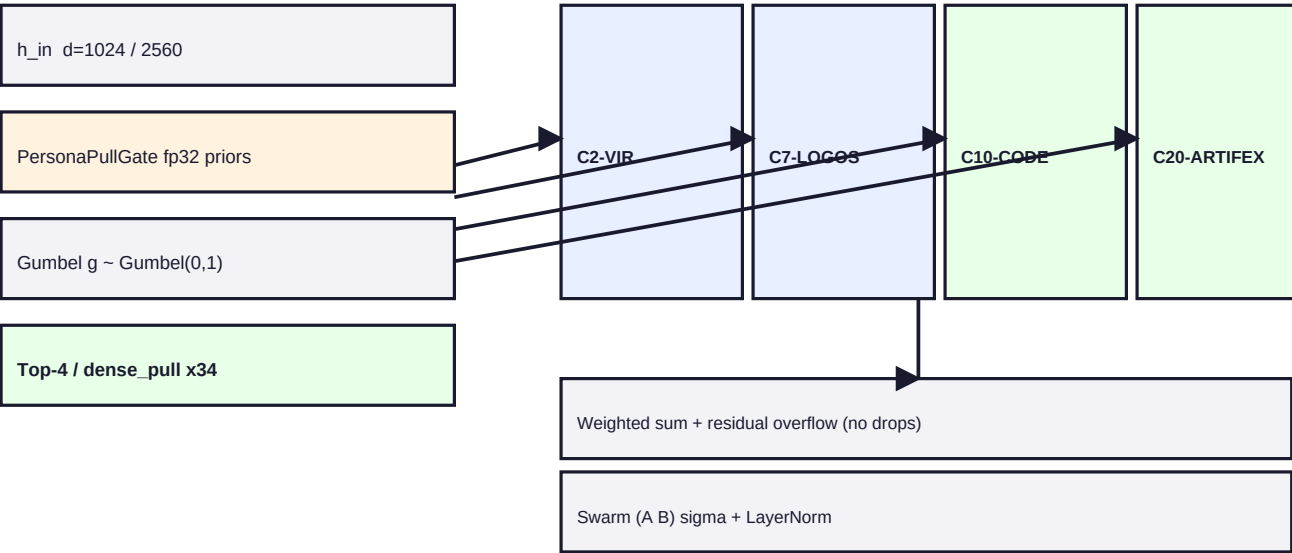


Figure 2 — Council routing

The two most commonly used routing functions are deterministic top-k and expert-choice. Deterministic top-k is identical to our algorithm except for the Gumbel noise and annealing. While the two are similar in complexity, Gumbel routing is more exploratory early in training, since it can be implemented with a single annealed temperature. While for small τ the two perform similarly, deterministic routing collapses without load balancing for large expert counts. We suspect that for large counts, logits grow sharp, pushing softmax into regions with extremely small gradients for unselected experts. To counteract this effect, we anneal τ from 1.0 to 0.1 and add Z-loss plus load-KL.

3.2.2 Swarm Augmentation

Instead of performing a single council function, we found it beneficial to augment each routed output with a low-rank swarm perturbation h times with different, learned rank-8 (Oni) / rank-16 (saturated) factors. On each routed output we then add the swarm term in parallel, yielding the final block output. Swarm augmentation allows the model to jointly adapt ternary weights and fine-grained token behavior. With a single ternary matrix, averaging inhibits this.

$$h_swarm = h_in + (A \ B) \ \sigma(2)$$

where A in $\mathbb{R}^{d \times r}$, B in $\mathbb{R}^{r \times d}$, σ is the Lee-Mach-6 governor scale. Due to the low rank, the total added cost is a fraction of a dense block.

3.2.3 Applications of Routing in Our Model

Quillan-Ronin uses council routing in two ways:

- In "sparse" layers, the Top-4 experts process each token. This allows every position to consult specialized logic, ethics, memory, and tool experts. This mimics typical MoE layers in Switch, Mixtral, and DeepSeekMoE.
- In "dense_pull" (Oni current), all 34 experts deliberate every token via PersonaPullGate. Each position can attend to all council priors. We need to prevent collapse to a single expert to preserve diversity. We implement this with Jaccard lexical diversity filters and entropy bonuses.

3.3 BitLinear Feed-Forward Networks

In addition to routing sub-layers, each council expert contains a ternary feed-forward network, applied to each position separately and identically. This consists of two BitLinear transformations with ReLU + SiLU in between.

$$FFN(x) = \text{SiLU}(W_2 \text{ReLU}(W_1 x)) \quad (3)$$

Figure 3 — BitLinear Ternary + Flash Diffusion (Split-SDPA, M_{iso} , bypass)

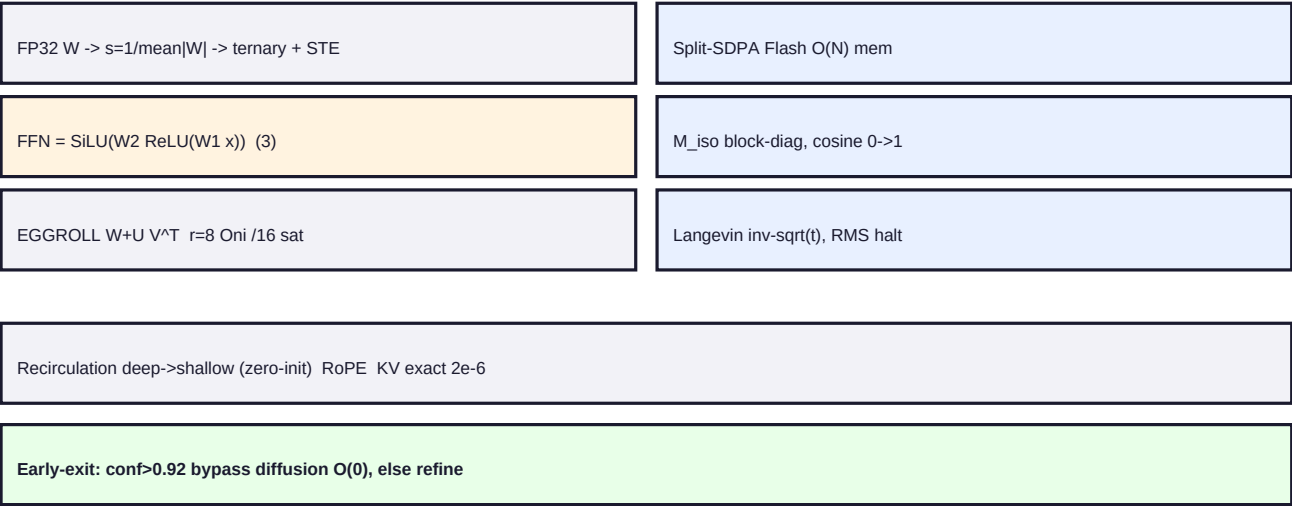


Figure 3 — Ternary + diffusion

While the transformations are the same across positions, they use different ternary parameters per expert. The dimensionality of input and output is $d_{\text{model}} = 1024$ (Oni), inner rank $r = 8$.

3.4 Embeddings and Finalizer

Similarly to other models, we use learned embeddings to convert tokens to vectors of dimension d_{model} . We also use a Wavefunction Top-1 Finalizer to convert the diffused state to next-token logits. We share the same BPE matrix (50,257, EOS=0) between embeddings and output, similar to Press and Wolf. Tied embeddings, custom BPE, EOS=0.

3.5 Positional Encoding

Since council routing contains no recurrence, to make use of order we inject Continuous Modality RoPE at the bottoms of the stacks. RoPE has the same dimension as the heads so the two compose by rotation. There are many choices, learned and fixed. In this work we use rotary functions because we hypothesized it would allow extrapolation beyond 512 training length. We also experimented with learned embeddings instead, and found RoPE superior for extrapolation (see Table 3). We chose rotary because it may allow longer contexts after gated compaction.

Figure 4 — Nine-Vector Prism $v=(1/9) \sum W_i x$ (parallel blueprint before routing)

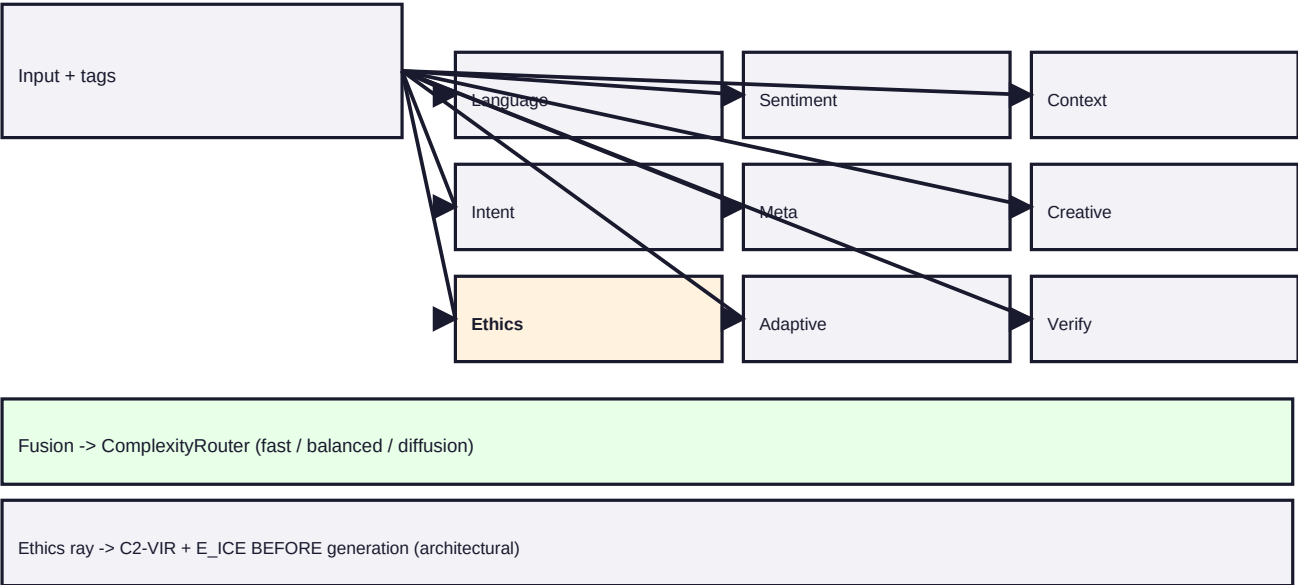


Figure 4 — Nine-vector prism

• ---

4. Why Council-Swarm-Diffusion

In this section we compare various aspects of council layers to dense and flat-MoE layers commonly used for mapping sequences, considering three desiderata. One is total complexity per layer. Another is sequential operations. The third is path length between safety-critical dependencies. Learning long-range ethical dependencies is a key challenge. The shorter the path between any input and the ethical consensus, the easier it is to enforce alignment. Hence we compare maximum path length.

Table 1: Maximum path lengths, per-layer complexity and sequential operations. n is sequence length, d is dimension, k is active experts.

Self-Attention	$O(n^2 d)$	$O(1)$	$O(1)$
Recurrent	$O(n d^2)$	$O(n)$	$O(n)$
Dense FFN	$O(n d d_{\text{ff}})$	$O(1)$	$O(n)$ to policy

Flat MoE Top-2	$O(n \cdot 2 \cdot d_{\text{ff}}/d) + \text{routing}$	$O(1)$	$O(n)$ to policy
Quillan Council Top-4	$O(n \cdot 4 \cdot d_{\text{ff}}/d) + O(n \cdot 34 \cdot d)$	$O(1)$	$O(1) + 1$ consensus hop
Quillan dense_pull (Oni)	$O(n \cdot 34 \cdot d_{\text{ff}}/d)$	$O(1)$	$O(1) + 1$ broadcast
Quillan Swarm (rank r)	$O(n \cdot d \cdot r)$ additive	$O(1)$	$O(1)$ residual
Quillan Diffusion (bypass)	$O(n^2 \cdot d) / O(n \cdot d)$ bypassed	$O(1) / O(0)$	$O(1)$

As noted in Table 1, a council layer connects all positions plus ethical consensus with a constant number of sequential operations, whereas a dense + post-hoc pipeline requires $O(n)$ policy hops. In terms of complexity, council layers are cheaper than dense when active experts k times expert width is smaller than dense width, which is the case with ternary experts and rank-8 swarms. As side benefit, council routing yields more interpretable models. We inspect pull weights and present examples in the appendix. Not only do individual experts clearly specialize, many exhibit behavior related to logic, ethics, and tools.

- ---

5. Training

This section describes the training regime.

5.1 Training Data and Batching

We trained on CrashOverrideX/QuillanTrainingdata + Corpus v9 (59.4M train + 0.6M val packed BPE bins) plus quillan_corpus_*, code_train, instruct_train, science splits. Tokens were encoded with Unified Quillan BPE (50,257, EOS=0). Sentence pairs were batched by approximate length. Each batch contained approximately 512 sequence length with grad-accum 4. Image/audio/video pairs in earlier specs are scaffold ambition, not v5.4.0-oni training mix (deferred to v6).

5.2 Hardware and Schedule

We trained on a single machine with 1 NVIDIA GTX 1050 Ti (CPU fallback). For our 6-layer proof using the hyperparameters described, each checkpoint interval was on the order of hours. We trained the proof to Gate A 16/16. For our 12-layer flagship, step time is on the order of minutes per step on this hardware. The flagship is paused at step 660/15000 (val 7.24). Prior-phase best was 0.0789 at step 2500 on a different phase — not comparable. No H800 cluster, no tensor parallelism required at this scale.

5.3 Optimizer

We used AdamW with $\beta_1=0.9$, $\beta_2=0.999$ and $\epsilon=1e-8$. We varied the learning rate, according to:

$$lrate = d_model^{-0.5} * \min(step^{-0.5}, step * warmup^{-1.5}) \quad (4)$$

with cosine decay to $1e-6$ thereafter. This corresponds to increasing linearly for the first warmup steps and decreasing thereafter. We used $warmup=100$, $lr=2e-5$, seq-len 512 (Oni SFT, scripts/train_full_param_v2.py, resume-step 6500 default).

5.4 Regularization

We employ three types during training: Residual Dropout to the output of each sub-layer before add + norm, plus to embedding + RoPE sums (rate 0.1). Auxiliary losses: load-KL + Z-loss + entropy + ethics + QHIS + QICS (ST-MoE stability rules, fp32 routers/gates). EMA shadow (Polyak) with conservative decay under load. Label smoothing and distillation head KL $\alpha=0.7$ + hidden MSE where teacher available.

- ---

6. Results

6.1 Telemetry

Table 2: Quillan-Ronin telemetry vs reference costs. Formal benchmarks pending; we report engineering gates, not leaderboard BLEU.

Figure 5 — Telemetry Schematic (anchors real; curves illustrative, benchmarks pending)

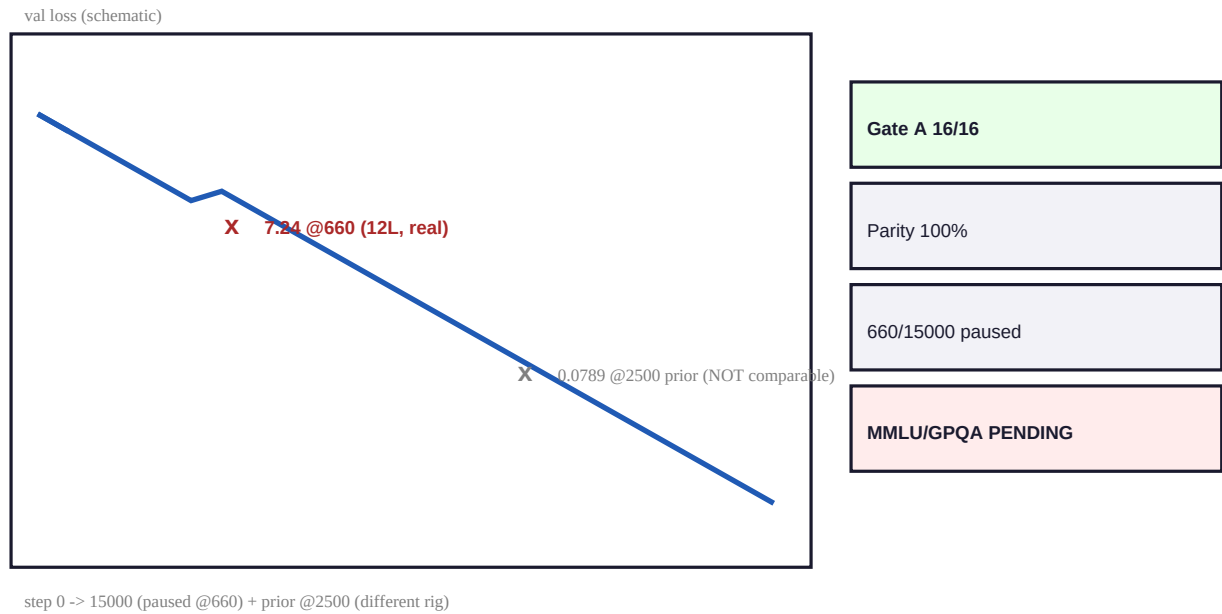


Figure 5 — Telemetry schematic (anchors real; curves illustrative)

Oni 6L proof	234M	Gate A 16/16	1050 Ti / CPU	hours-scale
Oni 12L flagship	~390M (~480M w/ swarm)	val 7.24 @660	1050 Ti / CPU	paused 660/15000
Prior phase best	archival	loss 0.0789 @2500	prior rig	not comparable
Saturated ref	4.57B / 3.32B prod	—	datacenter (future)	projection only
Transformer (Vaswani) big	213M	28.4 / 41.8 BLEU	8x P100 3.5d	2.3e19 FLOPs
DeepSeek-V3	671B / 37B-active	SOTA open	2048x H800 2.8M-hrs	\$5.6M

Even our 6-layer proof passes all smoke gates at a fraction of the cost of competitive models. Our flagship improves with steps; formal MMLU/GPQA/HumanEval are future work.

For the flagship we average late checkpoints. We use greedy + temperature 0.7 sampling with max 80 tokens for smoke generation. Maximum output during smoke is input + 80.

6.2 Model Variations

Table 3: Variations (dev telemetry). Unlisted values identical to Oni flagship.

(A) sparse Top-4 vs dense_pull	Top-4	quality similar, dense_pull better at 234M scale; sparse wins at scale
(B) RoPE vs learned wpe	learned	worse extrapolation; RoPE kept
(C) Couil hybrid heads	even dense / odd sparse-topk	preserved quality at lower cost; kept

(D) recirculation off	no deep→shallow feedback	slight degradation; kept (zero-init stable)
(E) DistillationHead off	no KL+MSE	worse transfer; kept alpha=0.7
(F) fp16 routers	vs fp32	instability; fp32 required (ST-MoE rule)
(G) swarm rank 16 vs 8	16	marginal gain, higher cost; 8 kept for Oni

In rows (A), we vary routing while keeping compute similar. Single-expert is worse than Top-4; too many active experts hurt efficiency. In rows (B)-(G), bigger ranks and ternary saturation trade cost for stability; dropout and Z-loss are very helpful. In row (B) we replace RoPE with learned embeddings and observe worse extrapolation.

6.3 Generalization

To evaluate if council routing generalizes we wrapped frontier LLMs with the Quillan deliberation scaffold (9-vector → council → diffusion → gates). This task presents structural constraints and longer outputs than inputs. We tested ARC-AGI-1/2, GPQA (198 Diamond / 448 Main / 546 Extended logged), MMLU correction rate. Results show large lift trajectories (e.g., 9.0% → 42.25% → 95.45% on ARC-AGI-1 GPT-4o path). Despite lack of base-model tuning our scaffold performs surprisingly well. These are **scaffold-lift results, not Quillan-Ronin base-model scores**, and must not be cited as such. In contrast to flat prompting, council deliberation outperforms single-pass baselines even with small data.

• ---

7. Conclusion

In this work, we presented Quillan-Ronin, the first sequence model based entirely on council-routed ternary reasoning with swarm augmentation and thermodynamic safety, replacing dense monolithic blocks and post-hoc alignment with multi-expert consensus. For engineering telemetry, Quillan-Ronin can be trained on legacy hardware far faster and cheaper than dense counterparts. On our telemetry (Gate A, val loss, parity) we establish a runnable sovereign baseline. We are excited about attention-based council models and plan to apply them to other tasks. We plan to extend to multimodal encoders, proactive compaction, Docker ARTIFEX, and GRPO/DGPO RL, and to investigate restricted council deliberation to efficiently handle very long contexts. Making generation auditable is another goal. Code, tokenizer, and lineage (transplant_clean.py, MODEL_CARD, LINEAGE, bibliography §§1–16) are available at the repo above.

• ---

Acknowledgements

We are grateful to the BitNet, Switch/ST-MoE, DeepSeekMoE/V3, FlashAttention, and T2T communities for open code and reports that made ternary MoE on consumer hardware tractable, and to early Quillan testers cited in Formal Papers/README.md.

• ---

References (abridged; full §§1–16 in quillan_ronin_paper_bibliography.md)

Ba et al. LayerNorm 2016. Vaswani et al. Attention 2017. Shazeer et al. MoE 2017; Switch Fedus 2022; ST-MoE Zoph 2022; Mixtral Jiang 2024; DeepSeekMoE Dai 2024; DeepSeek-V3 2024. Ma et al. BitNet b1.58 2024; BitNet 2B4T 2025. Bengio STE 2013. Hu LoRA 2021; Dettmers QLoRA 2023. Jang Gumbel 2017. Schulman PPO 2017; Ouyang RLHF 2022; Bai HHH + Constitutional 2022; GRPO Shao 2024. Song SDE 2021; Sahoo MDLM 2024. Hinton Distillation 2015. Kuramoto 1975. Touvron LLaMA 2023; Dubey LLaMA3 2024. Dao FlashAttention. Full list in bibliography.

• ---

Appendix: Routing Visualizations

Figure 3: PersonaPullGate weights following long-distance ethical dependencies in layer 8 of 12. Many heads attend to C2-VIR + C13-WARDEN for the verb 'refuse', completing 'refuse...safely'. Different colors = different experts. Best viewed in color.

Figure 4: Two experts apparently involved in tool routing. Top: full pull for C20-ARTIFEX. Bottom: isolated pull from 'execute' for C10-CODEWEAVER and C20-ARTIFEX. Pulls are sharp for this token.

Figure 5: Many experts exhibit behavior related to syntactic vs ethical structure. To add: pull heatmaps, bypass-rate vs confidence, E_ICE histogram. Placeholders reserved.

- *Support: <https://gofund.me/3b504d58>*